Conceptos básicos:

- Programa: Secuencias de instrucciones ordenadas y detalladas para dirigir la actuación y realización de operaciones de un computador electrónico.
- Computadora: Máquina electrónica que trata automáticamente la información y ejecuta procesos lógicos a gran velocidad.
- Programar: Elaborar un programa informático codificando las órdenes (escritas en algún lenguaje de programación) para resolver algún tipo de problema.

La computadora es una máquina compuesta por un conjunto de circuitos electrónicos, componentes mecánicos e interfaces para interactuar con el exterior (usuarios y otros dispositivos) cuya finalidad es procesar información para obtener resultados.

Los datos que constituyen la entrada a la computadora se procesan mediante una lógica (programa) para producir una salida.

El programador:

Persona responsable de indicar a la computadora la lógica de procesamiento que debe aplicarse sobre las entradas, expresada en forma de programa, para obtener los resultados.

Un programador debe:

- Analizar la cuestión a resolver (Problema o requerimiento).
- Tomar decisiones para ese fin (Estrategia).
- Diseñar el proceso (Algoritmo).

Algoritmo: Conjunto finito y ordenado de pasos que, ejecutados de manera correcta, permite obtener un resultado o resolver un determinado problema.

Problema: Situación que se nos presenta y que mediante la aplicación de un algoritmo pretendemos resolver.

Para diseñar un algoritmo para resolver un problema hay que estudiar el contexto del problema.

Computadoras:

Deben realizar un conjunto mínimo de funciones y proporcionar capacidad para:

1. Aceptar entradas.
2. Mostrar salidas.
3. Almacenar información en un formato lógico consistente (tradicionalmente binario).
4. Ejecutar operaciones aritméticas y lógicas en los datos de entrada o en los almacenados.
5. Supervisar, controlar y dirigir la operación y secuencia general del sistema.

“Computadora” se emplea para referir a dos partes:

- Hardware: la computadora en sí, junto a los dispositivos de entrada y salida.
- Software: conjunto de programas que controlan y dirigen la computadora para que realice tareas especificadas.

Almacenamiento:

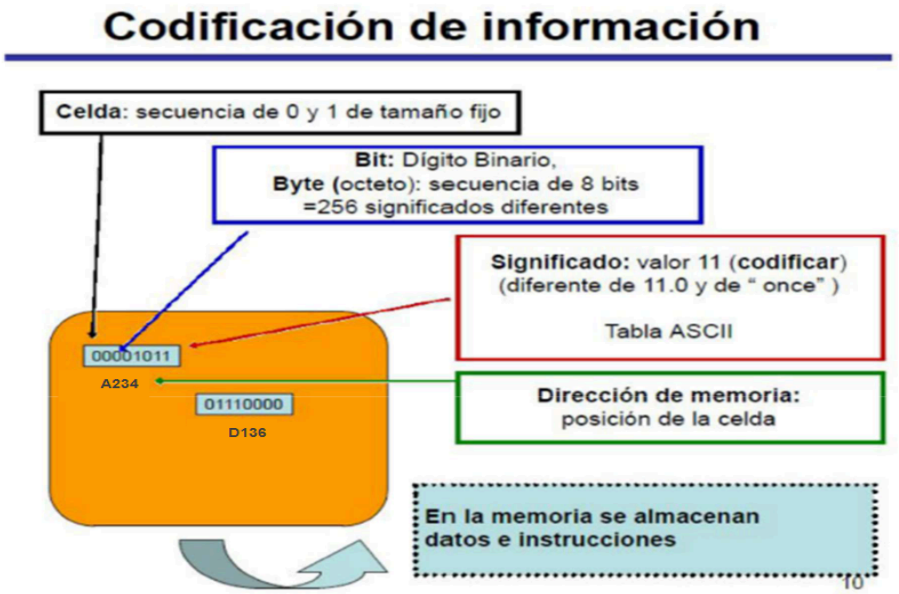
- Las computadoras sólo entienden un lenguaje compuesto de ceros y unos (Sistema Binario Digital).

- La unidad de información más pequeña que una computadora puede tratar se denomina bit (binary digit).
- La memoria central es una zona de almacenamiento organizada en celdas, cuya unidad mínima de información es el bit y al conjunto máximo de estos que la computadora es capaz de manejar en un momento dado se le denomina palabra.
- Cada celda tiene una dirección única que indica su posición relativa en la memoria y los datos que se almacenan en ella constituyen el contenido de la celda.
- La palabra de 8 bits se conoce como Byte (B) y a las otras se las conoce como palabras de 2 bytes (16 bits), de 4 bytes (32 bits), etc.
- Otras medidas de almacenamiento:

Kilobyte (KB) = 1024 B
Megabyte (MB) = 1024 KB
Gigabyte (GB) = 1024 MB
Terabyte (TB) = 1024 GB
Petabyte (PB) = 1024 TB

Memoria y almacenamiento:

- Los bytes sirven para representar caracteres en un código estándar internacional ASCII (American Standard Code for Information Interchange), base y extendido.
- La computadora dispone de dos tipos de memoria:
  - RAM (Random Access Memory): almacenamiento temporal de datos y programas. Es volátil. Muy rápida pero limitada en tamaño. En ella se almacenan datos de entrada para procesarse, programas que realizarán los procesos y resultados de salida.
  - Externa o secundaria o auxiliar: reside en dispositivos y es permanente.



Lenguajes de programación:

- Lenguajes de Bajo Nivel: lenguaje de máquina (programas ejecutables): 11000000 000000000001…
- Lenguajes de Alto Nivel: (Visual Basic, C, C++, Java, Python, etc). Necesitan traducirse a código de máquina a través de compiladores.

C++:

- El lenguaje C es un lenguaje de procedimiento estructurado desarrollado en la década de los 70', pero se convirtió en un lenguaje dominante para aplicaciones de ingeniería en los 80'.
- A partir de C se creó C++ con el fin de tener un lenguaje de programación orientado a objetos.

Compilación y ejecución de un programa:

- El algoritmo se escribe mediante un programa editor y se almacena en disco (archivo de programa), convirtiéndose en programa fuente.
- El programa es traducido a lenguaje de máquina por el compilador y el sistema operativo.
- Se realiza un montaje o enlace (link), es decir, una combinación del programa objeto con las bibliotecas del compilador. Esto produce un programa ejecutable (programa completo en lenguaje de máquina).
- El ejecutable se puede ejecutar o correr (run). Si no hay errores de ejecución, se obtendrá la salida del programa.

Entornos de programación (IDEs):

Para obtener el programa ejecutable necesitamos dos herramientas básicas:

- Un editor con el que crear un archivo con el código fuente.
- Un compilador para traducir este código fuente a código ejecutable.

Para realizar estos pasos podemos:

- Usar la línea de comandos del sistema operativo.
- Usar un IDE.

C++: Lenguaje compilado:

Antes de poder correr el programa que escribimos, tenemos que compilar el código fuente (archivo.cpp) con un compilador de C++.

El compilador lee y analiza el programa. Muestra los errores (si los hay) y si no traduce el programa a lenguaje de máquina.

El archivo comienza con unas líneas para incluir las definiciones de los módulos de biblioteca que utilice nuestro programa, e irá seguido de declaraciones y definiciones de tipos, de constantes y de subprogramas. El programa debe contener un subprograma especial (la función main) que indica dónde comienza la ejecución. Las instrucciones contenidas en dicha función main se ejecutarán una tras otra hasta llegar a su fin.

Creación y ejecución de un programa (etapas):

- Definir el programa
- Definir directivas del preprocesador

- Definir declaraciones globales
- Crear main()
- Crear el cuerpo del programa
- Crear las propias funciones definidas por el usuario
- Utilizar comentarios adecuados
- Almacenar en disco con nombre.cpp
- Compilar, enlazar, ejecutar y comprobar el programa

Directivas del Preprocesador:

Preprocesador: editor de texto inteligente que procesa directivas que son instrucciones al compilador antes de que se compile el programa principal.

Las dos directivas más usadas son:

- #include: indica al compilador que lea del disco el archivo fuente (de texto ASCII) que sigue, y su contenido lo inserte en esa posición.
- #define: indica al compilador que defina un ítem de datos u operación (ej: #define PI 3.1415) sustituirá PI por el valor 3.1415 cada vez que aparezca en el programa.

Elementos básicos de un programa:

Bibliotecas: C++ consta con de un reducido número de instrucciones, pero ofrece un amplio repertorio de bibliotecas con herramientas que pueden ser importadas por los programas cuando son necesarias. Por este motivo, un programa suele comenzar por tantas líneas #include como bibliotecas se necesiten.

Constantes: Podemos definir un nombre para referirnos a un valor para que el programa sea más legible y fácil de modificar en el futuro. Antes de utilizar una constante simbólica es necesario informar al compilador, indicando de qué tipo es, su nombre (identificador) y el valor que tiene asociado.

Identificadores: Para cada elemento que introduzcamos al programa debemos definir un nombre (identificador) con el que hacer referencia al mismo y disponer de algún mecanismo para informar al compilador de dicho nombre y de sus características.

- Un identificador debe estar formado por una secuencia de letras y dígitos en la que el primer carácter debe ser una letra. El carácter '_' puede utilizarse, sin embargo, los nombres no deben comenzar con dicho carácter.

Palabras reservadas: Algunas palabras tienen un significado especial en el lenguaje y no pueden ser utilizadas con otro sentido. ej: using, namespace, const, double, int, char, bool, void, for, while, do, if, switch, else, case, default, return, typedef, struct, break, entre otras.

Delimitadores: Símbolos que indican comienzo o fin de una entidad. ej: ( ) { } ; , < >

Operadores: Símbolos con significado propio según el contexto en el que se utilicen. ej: = << >> * / % + - < > <= >= == != ++ -- , entre otros.

Comentarios y formato del programa: Para que el compilador reconozca los distintos elementos del programa utiliza delimitadores y carácteres que permiten separar unos elementos con otros.

El programador puede añadir caracteres como espacios en blanco, líneas en blanco o tabuladores para mejorar la legibilidad del programa. El compilador los ignora. También puede agregar comentarios dirigidos a otros programadores los cuales también son ignorados por el compilador. Los tipos de comentarios son:

- Comentarios hasta fin de línea: Se marcan con los símbolos // y se extiende hasta el final de la línea.
- Comentarios enmarcados: Se marcan con los símbolos /* los cuales marcan el comienzo del comentario y se extiende hasta que se pone */ que determinan el final del comentario.

Variables: Los datos se almacenan en memoria en variables de un cierto tipo. El programador debe decidir qué variables va a utilizar, pensar en un identificador para referirse a ellas y comunicarlo al compilador.

Entrada / Salida de datos: C++ cuenta con flujos de entrada (cin) y de salida (cout), que nos permiten efectuar entrada y salida de datos. El operador >> se usa para tomar un valor de entrada y el operador << un valor de salida.

Asignación: El operador = separa el valor a almacenar, resultado de evaluar una expresión aritmética (a la derecha), de la variable en la que lo queremos almacenar (a la izquierda).

Depuración de un programa (proceso de encontrar errores). Los errores pueden ser:

- Errores de sintaxis: Se violan reglas de la gramática del lenguaje.
- Errores lógicos: Errores del programador en el diseño del algoritmo y posterior programa. No los detecta el compilador, se detectan con cuidadoso análisis de los resultados (testing). Se puede necesitar un depurador (debugger).
- Errores de ejecución: se presentan luego de que el programa fue compilado, durante la ejecución.
- Los compiladores emiten mensajes de error o de advertencia. Los mensajes de error del compilador suelen deberse a errores de sintaxis y pueden ser:
  - Fatales: no puede realizar la compilación.
  - De sintaxis: avisa y termina.
  - Warnings: indican condiciones legítimas pero sospechosas.

Pruebas:

El compilador puede no emitir ningún mensaje de error durante la ejecución, lo cual no garantiza que el programa sea correcto, sino que simplemente fue bien escrito sintácticamente. Para determinar si un programa tiene errores lógicos, se debe ejecutar utilizando datos de muestra y comprobar la salida (prueba-testing).

Si se ha determinado que el programa tiene un error lógico y no se lo encuentra fácilmente, para localizarlo se debe realizar la ejecución paso a paso, seguir la traza.

Máxima de Dijkstra (científico holandés): *"Las pruebas solo muestran la presencia de errores, no su ausencia. No se puede probar que un programa es correcto (exacto), solo se puede mostrar que es incorrecto."*

Tipos de datos en C++:

La elección de un determinado tipo u otro para una determinada entidad del programa proporciona información acerca de:

- El rango de posibles valores que puede tomar.
- El conjunto de operaciones y manipulaciones aplicables.
- El espacio de almacenamiento necesario para almacenar dichos valores.
- La interpretación del valor almacenado.

Los datos se clasifican en:

- Simples: valores indivisibles, es decir, no se dispone de operadores para acceder a parte de ellos. ej: int, float, double, char, bool.
- Compuestos: formados como un agregado o composición de otros tipos, ya sean simples o compuestos. ej: arreglos, cadenas, estructuras.

Simples predefinidos:

Tipo	Bytes	Bits	Min. Valor	Max. Valor
`bool`	1	8	`false`	`true`
`char`	1	8	-128	127
`short`	2	16	-32768	32767
`int`	4	32	-2147483648	2147483647
`long`	4	32	-2147483648	2147483647
`long long`	8	64	-9223372036854775808	9223372036854775807
`unsigned char`	1	8	0	255
`unsigned short`	2	16	0	65535
`unsigned`	4	32	0	4294967295
`unsigned long`	4	32	0	4294967295
`unsigned long long`	8	64	0	18446744073709551615
`float` (Precisión: 7 dígitos)	4	32	1.17549435e-38	3.40282347e+38
`double` (Precisión: 15 dígitos)	8	64	2.2250738585072014e-308	1.7976931348623157e+308
`long double`	12	96	3.362103143112093506 26e-4932	1.189731495357231765 02e+4932

Variables:

En C++ toda variable debe ser declarada antes de ser usada, dándole un identificador y asignándole un tipo.

Es posible acceder a cualquier variable declarada, pero si no se ha inicializado se referirá al valor "basura" que tiene almacenado el espacio de memoria.

Constantes:

Constantes definidas: son identificadores asociados a valores literales constantes. ej: #define PI 3.141592

Constantes declaradas: son como una variable pero su valor no puede ser modificado. ej: const int Meses = 12; const int Semana = 7; const float PI=3.141592

Expresiones:

Un programa se basa en la realización de una serie de cálculos con los que se producen los resultados deseados. Dichos resultados se almacenan en variables y a su vez son utilizados para nuevos cálculos, hasta obtener el resultado final.

Los cálculos se producen mediante la evaluación de expresiones en las que se mezclan operadores y operandos.

En caso de que en una misma expresión se utilice más de un operador habrá que conocer la precedencia de cada operador, y en caso de que haya varios operadores de igual precedencia, habrá que conocer su asociatividad.

↓ Cuadro con las reglas de precedencia abajo ↓

Reglas de Precedencia
(Ordenados de mayor a menor precedencia)

Mayor predencia

Menor predencia

Operador	Tipo de Operador	Asociatividad
[] -> .	Binarios	Izq. a Dch.
! ~ - *	Unarios	Dch. a Izq.
* / %	Binarios	Izq. a Dch.
+ -	Binarios	Izq. a Dch.
<< >>	Binarios	Izq. a Dch.
< <= > >=	Binarios	Izq. a Dch.
== !=	Binarios	Izq. a Dch.
&	Binario	Izq. a Dch.
^	Binario	Izq. a Dch.
|	Binario	Izq. a Dch.
&&	Binario	Izq. a Dch.
||	Binario	Izq. a Dch.
?:	Ternario	Dch. a Izq.

RESUMEN de Operadores C++

Operador	Descripción		Asociatividad
() [] -> .	Llamada a función Elemento de un arreglo Referencia al apuntador de miembro en una estructura Referencia a un miembro de una estructura		Izquierda a derecha
++ -- - ! ~ (type) sizeof & *	Incremento Decremento Menos unario Negación lógica Complemento a uno Conversión de tipo (molde) Tamaño del almacenamiento La dirección de Indirección		Derecha a izquierda
* / %	Multiplicación División Módulo (residuo)	Aritméticos	Izquierda a derecha
+ -	Adición Sustracción	Aritméticos	Izquierda a derecha
<< >>	Desplazamiento a la izquierda Desplazamiento a la derecha		Izquierda a derecha
< <= > >=	Menor que Menor que o igual a Mayor que Mayor que o igual a	Relacionales	Izquierda a derecha
== !=	Igual a Diferente a	Relacionales	Izquierda a derecha
&	AND bit por bit		Izquierda a derecha
^	OR exclusivo bit por bit		Izquierda a derecha
|	OR inclusivo bit por bit		Izquierda a derecha
&&	AND lógico	Lógicos	Izquierda a derecha
||	OR lógico	Lógicos	Izquierda a derecha
?:	Expresión condicional		Derecha a izquierda
= += -= *= /= %= &= ^= |= <<= >>=	Asignación Asignación Asignación Asignación Asignación		Derecha a izquierda
,	Coma		Izquierda a derecha

Operadores pre y post fijo:

Cuando el operador ++ aparece antes de una variable se llama operador de prefijo para incremento, mientras que cuando aparece después se llama operador de postfijo para incremento (lo mismo para los operadores --, solo que serán de decremento).

Para valor = ++n;

```
n=n+1; //primero incrementar n
valor = n; //luego asignar el valor de n a valor
```

Para valor = n++;

```
valor = n; //primero asignar el valor de n a valor
n = n+1; //y luego incrementar n
```

Operadores relacionales: == != < > <= >=

- Las expresiones y el operador deben ser compatibles
- Sirven para comprobar una condición
- Devuelven 0 (condición falsa) o 1 (condición verdadera)
- Los caracteres se comparan utilizando el código ASCII
- La comparación alfabética de cadenas debe hacerse con la función strcmp (cad1, cad2).

Operadores lógicos: ! (not) && (and) || (or)

- Las evaluaciones se corresponden con el álgebra de Boole y con sus prioridades.
- Se trabaja con evaluación en cortocircuito: se evalúa el operando de la izquierda y si éste determina en forma unívoca la expresión, el de la derecha no se evalúa.

Conversión de tipos:

Frecuentemente se necesita convertir el valor de un tipo de dato a otro sin cambiar el valor que representa.

Se hacen conversiones automáticas cuando:

- Se asigna un valor de un tipo a una variable de otro tipo.
- Se combinan tipos mixtos en expresiones.
- Se pasan argumentos a funciones.

Promoción de tipos: El tipo de dato que viene primero en esta lista se convierte al que viene segundo. char, int, unsigned int, long, unsigned long, float, double.

Conversión implícita: Cuando los tipos básicos están mezclado en asignaciones y expresiones. Los tipos de datos más bajos o con menos rango se convierten en los más altos o más rango.

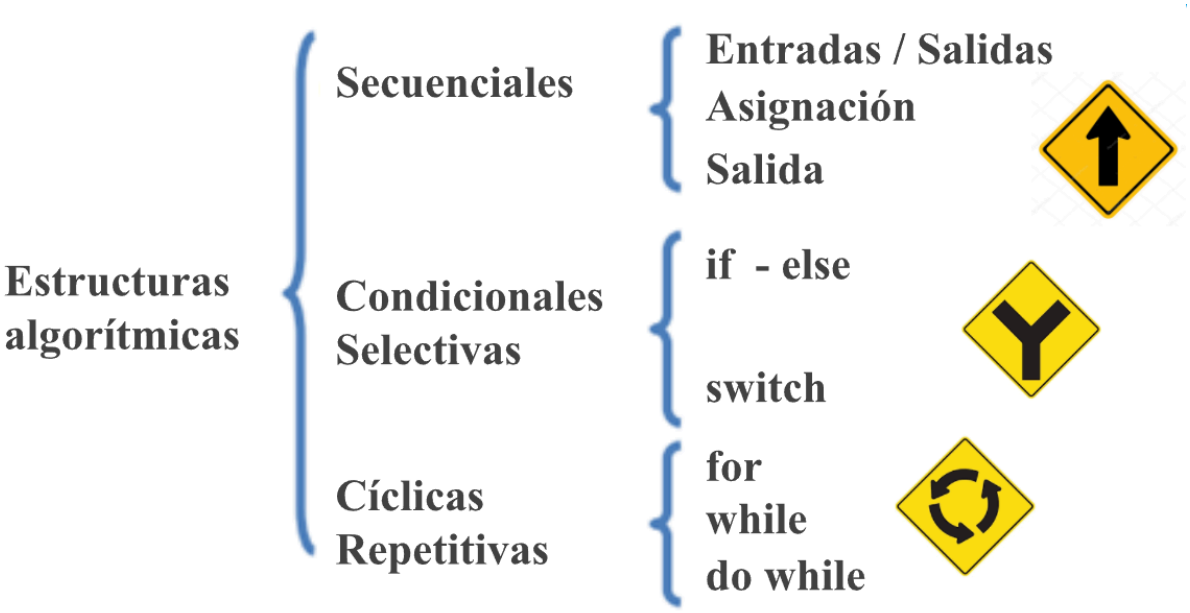
Conversión explícita: Cuando se fuerza la conversión con un operador de conversión de tipos (cast). ej: (float) valor (con valor siendo una variable de tipo int).

Algoritmos. Características:

- Eficaz: Debe solucionar el problema.
- Finito: Debe tener un fin, es decir, debe terminar su ejecución luego de una cantidad finita de pasos.
- Definido: Cada paso de un algoritmo no debe contener ambigüedades. Las acciones a seguir deben ser específicas.
- Preciso: Deben indicar qué pasos deben seguirse y en qué orden deben realizarse.

- Eficiente: El consumo de recursos debe ser aceptable. Muchas veces depende de la cantidad de datos a ser procesados, o bien de la forma en que los datos están organizados.
- Legible: El texto que lo describe debe ser claro, tal que permita entenderlo y leerlo fácilmente.

Estructuras de control:



Sentencia IF:

- Selección incompleta: Si sólo se desea realizar una acción en caso de que la condición sea verdadera y no hacer nada en caso de falso (no hay else).
- Selección completa: Se evalúa la expresión booleana y según sea verdadera o falsa, ejecutar una acción u otra.
- Selección anidada: Una sentencia if es anidada cuando en el cuerpo contiene otra sentencia if.

Sentencia SWITCH:

- Contiene etiquetas que evalúan casos literales (valores únicos y constantes) de la variable.
- Se van evaluando las etiquetas hasta que una coincide con el valor de la variable.
- Si el valor del selector no coincide con ningún caso no se ejecuta ninguna opción, a no ser que se use el default.